

CONSTRSA: A MINIMAL CONSTANT-TIME RSA-512 DIGITAL SIGNATURE WITH STATISTICAL TIMING-LEAK VERIFICATION ON STANDARD LAPTOPS

Research proposal

*Submitted in partial fulfilment of the requirement for the degree of Bachelor of
Science Honours in Information Technology*

By:
R.M.S.Rathnayake
2020/ICT/47

**Department of Physical Science
Faculty of Applied Science
University of Vavuniya
Sri Lanka**

December, 2025

1 Declaration

I, **Rathnayakage Maleesha Shehari Rathnayake** (2020/ICT/47), do hereby declare that the work reported in this project thesis was carried out independently by me under the supervision of [Mr. B. Yogarajah ,Senior Lecturer Gr.I,Department of Physical Science, Faculty of Applied Science, University of Vavuniya] , [Dr.S.Kirushanth , Senior Lecturer Gr.I,Department of Physical Science, Faculty of Applied Science, University of Vavuniya] and [Mr. K. Mathanaharan,Lecturer (Prob.),Department of Physical Science, Faculty of Applied Science, University of Vavuniya]. This thesis describes the results of my own research, except where due reference has been made in the text. No part of this research project has been submitted earlier or concurrently for the same or any other degree.

Date: _____

Signature: _____

Certified by:

1. Supervisor: Mr. B. Yogarajah

Date: _____

Signature: _____

2. Supervisor: Dr.S.Kirushanth

Date: _____

Signature: _____

3. Supervisor: Mr. K. Mathanaharan

Date: _____

Signature: _____

Approved by Coordinator/ IT4216

Name:Mr. T. Jeyamugan

Date: _____

Signature: _____

Approved by Head/ Department of Physical Science

Name:Mr. N. Edwin Linosh

Date: _____

Signature: _____

2 Introduction

2.1 Background

RSA-based digital signatures remain a foundational primitive in practical security systems, but real-world implementations frequently expose side-channel leakage when execution time or microarchitectural behavior depends on secret data, which enables key-recovery attacks in realistic threat models [5, 6, 1, 4]. Recent work demonstrates that timing leakage can persist even on modern computing platforms and accelerators, including GPUs and trusted execution environments, where shared resources and optimizations create additional attack surfaces [6, 4]. At the same time, defensive guidance consistently emphasizes constant-time engineering (eliminating secret-dependent branches and secret-dependent memory access) as a baseline mitigation for timing and cache side channels [5, 4, 1]. However, many constant-time implementations in major libraries are large and difficult to audit or study at undergraduate level, which limits their usefulness for education, experimentation, and reproducible leakage evaluation [6, 4].

2.2 Problem Definition

The main problem is that we do not have a small and easy-to-understand RSA digital signature program that is clearly constant-time and can be tested on a normal laptop. Constant-time means the program should take almost the same time to run, even when it uses different secret key values. This is important because attackers can sometimes measure how long the signing operation takes and use those small time differences to guess parts of the private key.

Many existing RSA implementations are made for real-world use in big libraries. ~~They try to be very fast and work on many different devices. Because of that, they use many optimizations and complicated code. This makes it hard for students and researchers to check whether the code is really constant-time.~~ It is also hard to use those large libraries for simple learning, testing, and benchmarking.

Also, modern research shows that timing leaks can be very small and still be dangerous. Even tiny differences in execution time, cache behavior, or CPU effects can help an attacker recover secret information if they collect many measurements. Because of this, we need an RSA implementation that is small enough to review, easy to experiment with, and designed to avoid secret-dependent branches and memory access from the beginning

2.3 Research Aim and Questions

The overall aim is to design, implement, and statistically validate a minimal constant-time RSA-512 signature scheme (RSA-PSS with SHA-256) that is suitable for controlled experiments and teaching on commodity laptops. This project is explicitly positioned as an ed-

educational and evaluation artifact; RSA-512 is not intended for production security, but it is appropriate for fast iteration and measurable leakage testing in an academic environment [1].

The study is guided by the following research questions:

- **RQ1:** How can RSA-512 signing (RSA-PSS + SHA-256) be engineered to avoid secret-dependent control flow and secret-dependent memory access patterns in a compact C implementation?
- **RQ2:** Does the resulting implementation exhibit statistically significant timing leakage under a standard fixed-vs-random timing experiment design executed on commodity laptops [1]?
- **RQ3:** What is the performance overhead of constant-time engineering relative to a straightforward non-constant-time baseline under identical compilation and measurement conditions?

2.4 Research Objectives

Primary Objective: To design, implement, and statistically validate a minimal constant-time RSA-512 digital signature (RSA-PSS with SHA-256) with a target code size under 10 KB, suitable for testing on standard consumer laptops.

Specific Objectives:

1. Implement a branchless RSA-512 digital signature in C using CRT, Montgomery multiplication, and fixed-window exponentiation.
2. Ensure constant-time execution by eliminating secret-dependent branching and secret-dependent memory access patterns.
3. Collect large-scale timing traces and apply Welch’s t-test to evaluate whether there is statistically significant timing leakage correlated with secret key material (e.g., using a $p > 0.05$ criterion or an equivalent standard threshold).
4. Benchmark signing and verification performance and report latency/throughput statistics.
5. Publish the implementation, documentation, and evaluation scripts as open-source.

2.5 Motivation and Significance

Timing and **microarchitectural** side channels remain a practical concern across platforms, reinforcing the need for implementations that are explicitly engineered for constant-time behavior and are verifiable through measurement [1]. This proposal is motivated by the educational

and research value of a small, auditable constant-time RSA artifact that enables students to connect theory (side-channel threat models) to practice (implementation discipline and leakage testing) in a reproducible manner. By providing a minimal implementation and an end-to-end evaluation pipeline, the work supports clearer examiner review while aligning with empirical expectations in side-channel research [1].

3 Related Works

Reference	Context / Platform	Main Focus	Key Contribution and Relevance to ConstRSA
Luo et al. (TACO) [6]	GPU-accelerated RSA	Timing side-channel attack	Demonstrates practical timing leakage in GPU RSA implementations and discusses how optimization does not eliminate leakage. Motivates strict constant-time engineering and simple, auditable implementations for controlled evaluation.
Luo et al. (ICCAD) [5]	GPU acceleration of RSA	Timing vulnerability analysis	Shows GPU-accelerated RSA is vulnerable to timing attacks; supports the proposal’s argument that platform/optimization choices can introduce observable leakage. Reinforces the need for measurement-driven validation.
Kou et al. (DATE) [4]	Trusted Execution Environments (TEEs)	Cache side-channel (sliding window)	Analyzes cache side-channel attacks/defenses for sliding-window exponentiation in TEEs. Directly informs the proposal’s constant-time exponentiation requirement (no secret-dependent table lookups/access patterns).
Aldaya & Brumley (TCHES) [1]	Microarchitectural leakage settings	Single-trace attacks on RSA/ECDSA	Shows how limited leakage can enable practical key recovery, emphasizing that even small, subtle side channels matter. Supports adopting rigorous leakage evaluation and conservative constant-time design.
Doctoral thesis (MUNI) [3]	Academic synthesis (broad)	Side channels / implementation security	Provides extended background and conceptual framing on implementation-level security and evaluation practices, useful for strengthening the proposal’s threat model and methodology justification.
Ding [2]	RSA in digital signature context	Application overview	Provides contextual discussion of RSA digital signatures and motivation; supports the “why RSA signatures matter” part of the introduction (not used as the primary side-channel evidence).

Table 1: **Summary of prior work relevant to constant-time RSA and leakage evaluation.** The table highlights the platform, threat focus, and how each work motivates the ConstRSA design and testing strategy.

4 Research Gap

The literature demonstrates that timing and cache side channels can remain exploitable across modern platforms, including GPUs and TEEs, and that constant-time engineering is a critical mitigation direction [6, 4, 1]. However, many constant-time implementations used in practice are integrated into large cryptographic libraries, which increases complexity, reduces auditability, and limits accessibility for undergraduate study and controlled experimentation [6, 4]. In parallel, several works emphasize evaluation and attack feasibility, but fewer provide minimal, pedagogical RSA artifacts designed specifically for reproducible leakage testing on commodity laptops under a clearly stated threat model [5, 1].

Therefore, the gap addressed in this proposal is the absence of a compact, auditable, and reproducible constant-time RSA-512 signature implementation with an accompanying timing-leak verification workflow that can be executed without specialized hardware, while still being motivated by contemporary attacker models and results [4, 6].

5 Materials and Methods

5.1 Research Design

This study uses a build-and-test approach. First, a compact RSA-512 digital signature program (RSA-PSS with SHA-256) will be implemented using constant-time coding rules. Second, the program will be tested by measuring signing time many times on a normal laptop and checking whether the timing changes depend on secret key information.

Threat Model: It is assumed that an attacker can collect many timing measurements of the signing operation. If the implementation is not constant-time, the attacker may find a relationship between the timing values and secret key behaviour.

5.2 Materials / Tools

- A C compiler and build tools (such as GCC or Clang) to implement and compile the RSA signing and verification code.
- A standard laptop/PC with a high-resolution timer to record execution time.
- Python (or similar) to store timing results, process datasets, and run statistical analysis.
- A Git repository (GitHub) to save code, track changes, and share the final implementation and results.

5.3 Implementation Procedures

1. **Core arithmetic:** Implement RSA-512 modular arithmetic using Montgomery multiplication and modular reduction. The code will be written to avoid secret-dependent branching.
2. **Exponentiation:** Implement modular exponentiation using a fixed-window method with constant-time table access so that exponent bits do not change the control flow or memory access pattern.
3. **CRT signing:** Implement CRT-based signing and recombination using branchless logic and constant-time operations to prevent key-dependent behaviour.
4. **Padding and hashing:** Implement RSA-PSS encoding and integrate SHA-256 so that signature generation and verification follow the correct standard format for testing.
5. **Baseline version:** Implement a separate simple baseline (non-constant-time) signing version to compare speed and show the overhead introduced by constant-time protections.

5.4 Dataset

The dataset for this research is a **timing dataset**. It is **created** by running the RSA-512 signing function many times and recording how long each run takes.

The dataset will be saved as a **CSV file**. Each row will include:

- run number,
- input type (fixed or random),
- measured signing time (nanoseconds or CPU cycles).

This timing dataset will be used to check whether signing time changes depending on secret key behaviour.

5.5 Timing Measurement and Leakage Testing

Timing traces will be collected by running the signing function many times using two test cases: a fixed input set and random input set. The recorded times will be compared using a statistical hypothesis test to check whether timing differences are significant. If the test does not show a significant difference, it suggests that the timing is not dependent on secret data.

To improve reliability, timing experiments will be repeated in multiple runs using the same laptop, same compiler settings, and controlled conditions where possible.

5.6 Performance Evaluation

Performance will be measured for both implementations (constant-time and baseline). The evaluation will include signing time and verification time. Results will be reported using clear statistics such as average time, median time, and variance so that the cost of constant-time protection can be understood.

5.7 Workflow of the Proposed System

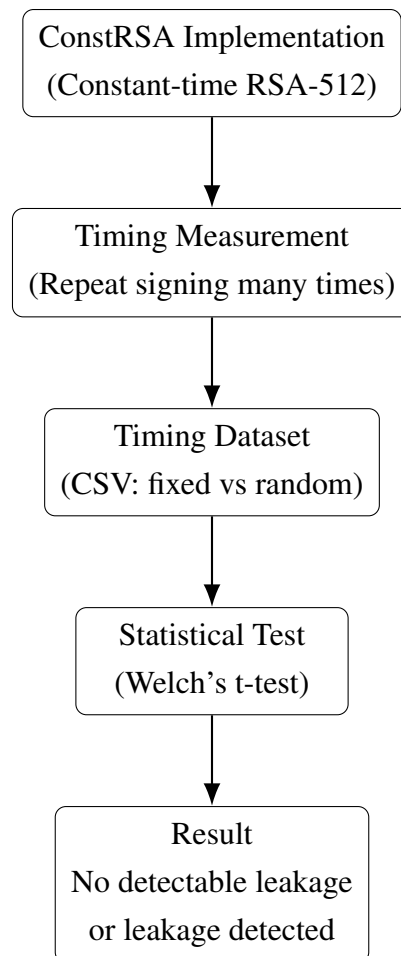


Figure 1: **End-to-end workflow of the proposed study.** The implementation is measured repeatedly to form a timing dataset, which is then analyzed using a statistical test to check for detectable timing leakage.

6 Expected Outcome

The expected outcomes of this research are:

1. A small and easy-to-check **constant-time RSA-512 digital signature implementation** (RSA-PSS with SHA-256), with a target size of **under 10 KB**, that can run on a normal laptop.
2. **Statistical results** from timing experiments showing whether the implementation has detectable timing leakage or not.
3. A public **open-source repository** that includes the source code, documentation, and testing scripts for students and researchers to use.
4. A contribution to **secure coding and cryptographic education** by providing a clear example of constant-time programming and timing-leak testing.

7 Ethical Approval

This research involves no human participants, no personal data, and no interaction with real-world production systems, and all experiments will be performed using self-generated RSA keys and locally collected timing traces [4]. The work focuses on defensive engineering and controlled evaluation of a toy-size RSA parameter set for research and teaching purposes, which reduces the risk of direct operational misuse while enabling the study of side-channel principles [1, 2]. Any released artifacts will include clear documentation on scope, limitations, and non-production intent, consistent with responsible disclosure norms in security research [6].

8 Gantt Chart

Work/Time (Months)	Nov-24	Dec-24	Jan-25	Feb-25	Mar-25	Apr-25	May-25	Jun-25
Title selection								
Proposal								
Literature Review								
Resource gathering								
Tools and Techniques								
Implementation								
Writing								
Publication / Conferences								
Presentation and Submission								

References

- [1] Alejandro Cabrera Aldaya and Billy Bob Brumley. “When one vulnerable primitive turns viral: Novel single-trace attacks on ECDSA and RSA”. In: *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)* 2020.2 (2020), pp. 196–221. DOI: 10.13154/TCHES.V2020.I2.196–221. URL: <https://d-nb.info/1206005157/34>.
- [2] Mashuo Ding. “Analyzing the Research and Application of the RSA Cryptosystem in the Context of Digital Signatures”. In: *Transactions on Computer Science and Intelligent Systems Research* 5 (2024), pp. 26–34. DOI: 10.62051/j2p6g984. URL: <https://wepub.org/index.php/TCSISR/article/view/2356>.
- [3] *Doctoral Thesis (source provided by user)*. https://is.muni.cz/th/rbeu1/doctoral_thesis.pdf. Provided as related work link; direct fetch timed out in this environment (access date: 2026-01-19).

- [4] Zili Kou et al. “Cache Side-channel Attacks and Defenses of the Sliding Window Algorithm in TEEs”. In: *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. DBLP record for bibliographic details. 2023, pp. 1–6. DOI: 10.23919/DATE56975.2023.10137116. URL: <https://dblp.org/rec/conf/date/KouSHZ23>.
- [5] Chao Luo, Yungsi Fei, and David R. Kaeli. “GPU Acceleration of RSA is Vulnerable to Side-channel Timing Attacks”. In: *Proceedings of the International Conference On Computer Aided Design (ICCAD)*. DBLP record for bibliographic details. 2018. DOI: 10.1145/3240765.3240812. URL: <https://dblp.org/rec/conf/iccad/LuoFK18>.
- [6] Chao Luo, Yungsi Fei, and David R. Kaeli. “Side-channel Timing Attack of RSA on a GPU”. In: *ACM Transactions on Architecture and Code Optimization* 16.3 (2019). DBLP record for bibliographic details, 32:1–32:18. DOI: 10.1145/3341729. URL: <https://dblp.org/rec/journals/taco/LuoFK19>.